

Phase 2: Open Hashing with Chaining — Full Structured Note

1. Current Progress Analysis

Before Phase 2, you already learned the foundation of hashing in Phase 1.

You learned that hashing tries to store and find data by calculating its position directly.

The basic process is:

Key → Hash Function → Index

Example:

$$H(15) = 15 \bmod 10 = 5$$

This means key **15** should be stored at index **5**.

But Phase 1 ended with an important problem: **collisions**.

A collision happens when two or more keys produce the same index.

Example:

15 mod 10 = 5
25 mod 10 = 5
35 mod 10 = 5

All three keys want index **5**.

A normal array position cannot directly hold many values at the same time.

So we need a method to solve this collision problem.

Phase 2 teaches the first collision-resolution method:

Open Hashing / Chaining

2. Main Goal of Phase 2

The main goal of Phase 2 is to understand how hashing handles collisions using linked lists.

In this phase, you learn:

- What open hashing means
- What chaining means
- What a bucket is
- How a hash table can be an array of pointers
- How linked-list chains store colliding keys
- Why chains are kept sorted
- How insertion works in chaining
- How searching works in chaining
- How early stopping works
- What loading factor means
- How time complexity works in chaining
- How deletion works in chaining
- Why a bad hash function can still make hashing slow
- How chaining is implemented in C
- Common beginner mistakes

The main beginner idea is:

If many keys go to the same index, store them in a linked list at that index.

3. What Problem Does Chaining Solve?

Suppose the table size is .

The hash function is:

$$H(x) = x \bmod 10$$

Now insert these keys:

15, 25, 35

Calculate their indices:

```
15 mod 10 = 5
25 mod 10 = 5
35 mod 10 = 5
```

All of them want index **5**.

Without collision handling, this is a problem:

```
Index 5: 15
```

Now where should **25** go?

Where should **35** go?

Chaining solves this by making index **5** point to a linked list:

```
Index 5: 15 -> 25 -> 35 -> NULL
```

So the collision is not ignored.

The colliding keys are connected together in a chain.

4. What Is Chaining?

Chaining is a collision-resolution method.

It means each index in the hash table can point to a linked list.

Each linked list stores all keys that hash to that same index.

Example:

```
Index 0: NULL
Index 1: NULL
Index 2: 12 -> 122 -> NULL
Index 3: NULL
Index 4: NULL
Index 5: 5 -> 25 -> 75 -> NULL
Index 6: 6 -> 16 -> NULL
Index 7: NULL
```

Index 8: 68 -> NULL
Index 9: 39 -> NULL

Here:

- Index 2 has two keys: 12 and 122
- Index 5 has three keys: 5, 25, and 75
- Index 6 has two keys: 6 and 16
- Empty indices contain NULL

Simple meaning:

Chaining means collisions become linked lists.

5. Why Is It Called Open Hashing?

Chaining is also called **open hashing**.

It is called open hashing because the hash table is allowed to use extra memory outside the original array.

The original array stores pointers.

The actual keys may be stored in linked-list nodes outside the array.

So the table is open to using extra memory.

Simple comparison:

Method	Meaning
Open hashing / chaining	Use extra linked-list memory outside the table
Closed hashing / open addressing	Keep all keys inside the original array

Phase 2 studies open hashing.

Later phases study closed hashing.

6. Bucket Meaning

Each position in the hash table is called a **bucket**.

A bucket is one place in the table where keys with the same hash index are stored.

Example:

```
Index 5: 5 -> 25 -> 75 -> NULL
```

Here, index **5** is a bucket.

The bucket contains a chain of keys.

Simple meaning:

```
Bucket = one hash table index that may contain a linked list.
```

7. Real-Life Analogy

Imagine a school has 10 classrooms:

```
Room 0, Room 1, Room 2, ..., Room 9
```

Students are assigned to rooms using:

```
student ID mod 10
```

Example:

```
Student 25 → 25 mod 10 = 5 → Room 5  
Student 75 → 75 mod 10 = 5 → Room 5  
Student 15 → 15 mod 10 = 5 → Room 5
```

All three students go to Room 5.

Instead of saying only one student can enter, we make a list inside Room 5:

```
Room 5: 15 -> 25 -> 75
```

That is chaining.

In hashing terms:

Real-life idea	Hashing idea
Classroom number	Hash table index
Student ID	Key
List of students in a room	Linked-list chain
Many students assigned to one room	Collision

8. Hash Table as an Array of Pointers

In Phase 1, the hash table looked like a simple array of values.

Example:

```
Index 0: 10
Index 1: empty
Index 2: empty
Index 3: 3
```

In chaining, the hash table is different.

Each array position stores a pointer to a linked list.

In C, this can be written as:

```
struct Node *HT[10];
```

This means:

```
HT[0] is a pointer to a linked list
HT[1] is a pointer to a linked list
HT[2] is a pointer to a linked list
...
HT[9] is a pointer to a linked list
```

At the beginning, every bucket is empty.

So every pointer is `NULL` :

```
for (int i = 0; i < 10; i++) {  
    HT[i] = NULL;  
}
```

Simple meaning:

NULL means this bucket currently has no chain.

9. Linked List Node Structure

A linked list is made of nodes.

Each node contains:

1. The data/key
2. A pointer to the next node

In C:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning:

Part	Meaning
data	Stores the key
next	Points to the next node in the chain

Example chain:

5 -> 25 -> 75 -> NULL

This means:

Node 1 data = 5, next points to 25
Node 2 data = 25, next points to 75
Node 3 data = 75, next points to NULL

`NULL` means the chain has ended.

10. Hash Function Used in Phase 2

The phase uses the simple modulus hash function:

$$H(x) = x \bmod 10$$

In C:

```
int hash(int key) {  
    return key % 10;  
}
```

Examples:

```
16 mod 10 = 6  
12 mod 10 = 2  
25 mod 10 = 5  
39 mod 10 = 9  
122 mod 10 = 2
```

The hash function tells us which bucket to use.

Then chaining tells us what to do inside that bucket.

11. Sorted Chains

In this phase, each linked list chain is kept sorted.

Sorted means the values are arranged from smallest to largest.

Example of a sorted chain:


```
5 -> 25 -> 75 -> NULL
```

Example of an unsorted chain:

```
75 -> 5 -> 25 -> NULL
```

The curriculum uses sorted chains because sorted chains make unsuccessful searches faster.

Example:

```
Index 5: 5 -> 25 -> 75 -> NULL
```

Search for 15.

Steps:

```
Check 5
15 is greater than 5, so continue.
```

```
Check 25
15 is smaller than 25, so stop.
```

Because the chain is sorted, if we already passed the possible position of 15, then 15 cannot appear later.

This is called the **early stop rule**.

12. Insertion in Chaining

Insertion means adding a new key to the hash table.

Insertion steps:

1. Take the key.
2. Calculate the index using the hash function.
3. Go to that bucket.
4. Insert the key into the linked list at that bucket.
5. Keep the chain sorted.

Example:

Insert key 25.

```
25 mod 10 = 5
```

Go to index 5.

If index 5 is empty:

```
Index 5: 25 -> NULL
```

If index 5 already has values:

```
Index 5: 5 -> 75 -> NULL
```

Then insert 25 in sorted order:

```
Index 5: 5 -> 25 -> 75 -> NULL
```

13. Full Insertion Dry Run

Use:

```
Table size = 10  
Hash function:  $H(x) = x \bmod 10$   
Keys: 16, 12, 25, 39, 6, 122, 5, 68, 75
```

Start with an empty table:

```
Index 0: NULL  
Index 1: NULL  
Index 2: NULL  
Index 3: NULL  
Index 4: NULL  
Index 5: NULL  
Index 6: NULL  
Index 7: NULL
```

Index 8: NULL
Index 9: NULL

Step 1: Insert 16

$16 \bmod 10 = 6$

Index 6 is empty.

Index 6: 16 -> NULL

Step 2: Insert 12

$12 \bmod 10 = 2$

Index 2 is empty.

Index 2: 12 -> NULL

Step 3: Insert 25

$25 \bmod 10 = 5$

Index 5 is empty.

Index 5: 25 -> NULL

Step 4: Insert 39

$39 \bmod 10 = 9$

Index 9 is empty.

Index 9: 39 -> NULL

Step 5: Insert 6

$6 \bmod 10 = 6$

Index 6 already has:

16 -> NULL

This is a collision.

Since chains must be sorted, insert 6 before 16:

Index 6: 6 -> 16 -> NULL

Step 6: Insert 122

$122 \bmod 10 = 2$

Index 2 already has:

12 -> NULL

This is a collision.

Since 122 is greater than 12, insert after 12:

Index 2: 12 -> 122 -> NULL

Step 7: Insert 5

$5 \bmod 10 = 5$

Index 5 already has:

25 -> NULL

This is a collision.

Since 5 is smaller than 25, insert before 25:

Index 5: 5 -> 25 -> NULL

Step 8: Insert 68

$68 \bmod 10 = 8$

Index 8 is empty.

Index 8: 68 -> NULL

Step 9: Insert 75

$75 \bmod 10 = 5$

Index 5 already has:

5 -> 25 -> NULL

This is a collision.

Since 75 is greater than 25, insert after 25:

Index 5: 5 -> 25 -> 75 -> NULL

Final Table After All Insertions

Index 0: NULL
Index 1: NULL
Index 2: 12 -> 122 -> NULL
Index 3: NULL
Index 4: NULL
Index 5: 5 -> 25 -> 75 -> NULL
Index 6: 6 -> 16 -> NULL
Index 7: NULL
Index 8: 68 -> NULL
Index 9: 39 -> NULL

Main lesson:

A collision does not replace the old key.
A collision adds the new key to the chain.

14. Insertion Dry Run Table

Key	Calculation	Index	Action
16	$16 \bmod 10$	6	Index 6 empty, create chain 16
12	$12 \bmod 10$	2	Index 2 empty, create chain 12
25	$25 \bmod 10$	5	Index 5 empty, create chain 25
39	$39 \bmod 10$	9	Index 9 empty, create chain 39
6	$6 \bmod 10$	6	Collision with 16, insert before 16: 6 -> 16
122	$122 \bmod 10$	2	Collision with 12, insert after 12: 12 -> 122
5	$5 \bmod 10$	5	Collision with 25, insert before 25: 5 -> 25
68	$68 \bmod 10$	8	Index 8 empty, create chain 68
75	$75 \bmod 10$	5	Collision in chain, insert after 25: 5 -> 25 -> 75

15. Searching in Chaining

Searching means checking whether a key exists in the hash table.

Search steps:

1. Take the search key.
2. Calculate the index using the hash function.
3. Go to that bucket only.
4. Search inside the linked list at that bucket.
5. If found, return success.
6. If not found, return failure.

Important:

You do not search the whole table.
You only search the correct bucket.

Example:

Search for 75.

$75 \bmod 10 = 5$

Go to index 5:

Index 5: 5 -> 25 -> 75 -> NULL

Search inside that chain:

Check 5
Check 25
Check 75
Found

16. Search Dry Run

Use the final table:

Index 0: NULL
Index 1: NULL
Index 2: 12 -> 122 -> NULL
Index 3: NULL
Index 4: NULL
Index 5: 5 -> 25 -> 75 -> NULL
Index 6: 6 -> 16 -> NULL
Index 7: NULL
Index 8: 68 -> NULL
Index 9: 39 -> NULL

Search for 12

$12 \bmod 10 = 2$

Go to index **2**:

12 -> 122 -> NULL

Check first node:

12 found

Result:

Found in 1 comparison.

Search for 75

75 mod 10 = 5

Go to index **5**:

5 -> 25 -> 75 -> NULL

Check:

5 is not 75
25 is not 75
75 is found

Result:

Found in 3 comparisons.

Search for 65

65 mod 10 = 5

Go to index **5**:


```
5 -> 25 -> 75 -> NULL
```

Check:

```
5 is not 65  
25 is not 65  
75 is greater than 65
```

Because the chain is sorted, stop.

Result:

```
65 not found.
```

Search for 15

```
15 mod 10 = 5
```

Go to index **5**:

```
5 -> 25 -> 75 -> NULL
```

Check:

```
5 is not 15  
25 is greater than 15
```

Because the chain is sorted, stop.

Result:

```
15 not found.
```

17. Search Dry Run Table

Search key	Hash index	Search path	Result
12	2	Check 12	Found in 1 comparison
75	5	Check 5, 25, 75	Found in 3 comparisons
65	5	Check 5, 25, then stop at 75 because 75 > 65	Not found
15	5	Check 5, then stop at 25 because 25 > 15	Not found

18. Early Stop Rule

The early stop rule works only because chains are sorted.

Suppose the chain is:

```
5 -> 25 -> 75 -> NULL
```

Search for 15.

Steps:

```
Check 5
15 is greater than 5, so continue.

Check 25
15 is smaller than 25, so stop.
```

Why can we stop?

Because if the list is sorted, all values after 25 are also greater than 25.

So 15 cannot be after 25.

Simple meaning:

In a sorted chain, once you pass the place where the key should be, you can stop.

19. Why Sorted Chains Matter

Without sorted chains, searching for a missing key may require checking the whole chain.

Unsorted chain:

75 -> 5 -> 25 -> NULL

Search for 15.

You cannot stop early, because the values are not in order.

You must check:

75, 5, 25

Sorted chain:

5 -> 25 -> 75 -> NULL

Search for 15.

You check:

5, 25

Then stop because $25 > 15$.

So sorted chains are useful especially for unsuccessful searches.

20. Loading Factor

The loading factor tells us how full the hash table is.

Formula:

$$\lambda = n / \text{size}$$

Where:

n = number of keys stored
 size = number of buckets in the hash table

In chaining, the loading factor also tells us the average chain length, if the hash function spreads keys evenly.

Example:

$n = 9$ keys
 $\text{size} = 10$ buckets
 $\lambda = 9 / 10 = 0.9$

This means the average chain length is about **0.9** keys per bucket.

Another example:

$n = 100$ keys
 $\text{size} = 10$ buckets
 $\lambda = 100 / 10 = 10$

This means the average chain length is about **10** keys per bucket.

Simple meaning:

Bigger λ usually means longer chains.
Longer chains usually mean slower searching.

21. Time Complexity in Chaining

Hashing has two parts:

1. Calculate the index.
2. Search or insert inside the chain.

Calculating the hash index is usually fast:

$O(1)$

But searching inside the chain depends on the chain length.

Average Search Cost

If keys are distributed evenly, the average chain length is:

$\lambda = n / \text{size}$

The formulas given in the curriculum are:

Operation	Average formula	Simple meaning
Hash calculation	$O(1)$	Calculating $\text{key \% size}$ is constant time
Successful search	$1 + \lambda / 2$	You usually check about half the chain
Unsuccessful search	$1 + \lambda$	You may check the whole chain

Beginner Explanation

If chains are short:

Search is fast.

If one chain becomes very long:

Search becomes slow.

So chaining is usually efficient when:

The hash function spreads keys evenly.

22. Example of Loading Factor and Search Cost

Suppose:

```
n = 20 keys  
size = 10 buckets
```

Then:

```
lambda = n / size  
lambda = 20 / 10  
lambda = 2
```

Average chain length is about 2.

Successful search average:

```
1 + lambda / 2  
= 1 + 2 / 2  
= 1 + 1  
= 2
```

Unsuccessful search average:

```
1 + lambda  
= 1 + 2  
= 3
```

Simple meaning:

```
On average, a successful search may check about 2 things.  
An unsuccessful search may check about 3 things.
```

23. Deletion in Chaining

Deletion means removing a key from the hash table.

Deletion in chaining is easier than deletion in linear probing.

Deletion steps:

1. Take the key to delete.
2. Calculate the index using the hash function.
3. Go to that bucket.
4. Search the linked list at that bucket.
5. If the key is found, unlink that node.
6. Free the deleted node from memory.

Example:

Delete 25.

$25 \bmod 10 = 5$

Go to index 5:

Index 5: 5 -> 25 -> 75 -> NULL

Remove 25 by changing the link from 5 to 75.

Before deletion:

5 -> 25 -> 75 -> NULL

After deletion:

5 -> 75 -> NULL

Simple meaning:

Deletion disconnects the node from the chain.

24. Deletion Cases

There are three common deletion cases.

Case 1: Delete the first node

Before:

```
25 -> 75 -> NULL
```

Delete 25.

After:

```
75 -> NULL
```

The head pointer must change.

Case 2: Delete a middle node

Before:

```
5 -> 25 -> 75 -> NULL
```

Delete 25.

After:

```
5 -> 75 -> NULL
```

The previous node must skip the deleted node.

Case 3: Delete the last node

Before:

```
5 -> 25 -> 75 -> NULL
```

Delete 75.

After:

```
5 -> 25 -> NULL
```


The previous node now points to `NULL`.

25. Bad Hash Function Example

A hash function should spread keys evenly.

But sometimes a bad hash function sends many keys to the same bucket.

Example keys:

5, 35, 95, 145, 175

Hash function:

$H(x) = x \bmod 10$

Calculations:

5 mod 10 = 5
35 mod 10 = 5
95 mod 10 = 5
145 mod 10 = 5
175 mod 10 = 5

All keys go to index `5`.

Final table:

Index 0: NULL
Index 1: NULL
Index 2: NULL
Index 3: NULL
Index 4: NULL
Index 5: 5 -> 35 -> 95 -> 145 -> 175 -> NULL
Index 6: NULL
Index 7: NULL
Index 8: NULL
Index 9: NULL

This is bad because one chain becomes very long.

The hash table starts behaving like a normal linked list.

Main lesson:

Chaining solves collisions, but a bad hash function can still make hashing slow.

26. Improving the Bad Hash Function

For the bad example:

5, 35, 95, 145, 175

Using the last digit with `mod 10` is bad because all keys end in `5`.

A better hash function may use another part of the number.

Example idea:

$H(x) = (x / 10) \bmod 10$

This uses the tens digit instead of the last digit.

Examples:

35 / 10 = 3, then 3 mod 10 = 3
95 / 10 = 9, then 9 mod 10 = 9
145 / 10 = 14, then 14 mod 10 = 4
175 / 10 = 17, then 17 mod 10 = 7

This spreads the keys better.

Simple rule:

Choose a hash function that matches your data pattern.

27. Complete C Program for Chaining

This program implements chaining using sorted linked lists.

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

struct Node {
    int data;
    struct Node *next;
};

int hash(int key) {
    return key % SIZE;
}

void sortedInsert(struct Node **head, int key) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = key;
    newNode->next = NULL;

    if (*head == NULL || key < (*head)->data) {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    struct Node *current = *head;

    while (current->next != NULL && current->next->data < key) {
        current = current->next;
    }

    newNode->next = current->next;
    current->next = newNode;
}

struct Node *search(struct Node *head, int key) {
    struct Node *current = head;

    while (current != NULL && current->data <= key) {
        if (current->data == key) {
            return current;
        }
    }
}
```

```

        current = current->next;
    }

    return NULL;
}

void deleteKey(struct Node **head, int key) {
    struct Node *current = *head;
    struct Node *previous = NULL;

    while (current != NULL && current->data < key) {
        previous = current;
        current = current->next;
    }

    if (current != NULL && current->data == key) {
        if (previous == NULL) {
            *head = current->next;
        } else {
            previous->next = current->next;
        }

        free(current);
    }
}

void insert(struct Node *table[], int key) {
    int index = hash(key);
    sortedInsert(&table[index], key);
}

void printTable(struct Node *table[]) {
    for (int i = 0; i < SIZE; i++) {
        printf("%d: ", i);

        struct Node *current = table[i];

        while (current != NULL) {
            printf("%d -> ", current->data);
            current = current->next;
        }

        printf("NULL\n");
    }
}

int main() {
    struct Node *HT[SIZE];

```

```

    for (int i = 0; i < SIZE; i++) {
        HT[i] = NULL;
    }

    int keys[] = {16, 12, 25, 39, 6, 122, 5, 68, 75};
    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++) {
        insert(HT, keys[i]);
    }

    printTable(HT);

    int searchKey = 75;
    struct Node *result = search(HT[hash(searchKey)], searchKey);

    if (result != NULL) {
        printf("Found %d\n", result->data);
    } else {
        printf("Key not found\n");
    }

    deleteKey(&HT[hash(25)], 25);

    printf("After deleting 25:\n");
    printTable(HT);

    return 0;
}

```

28. Code Explanation: struct Node

```

struct Node {
    int data;
    struct Node *next;
};

```

This creates a linked-list node.

Each node stores:

```
data = the key  
next = pointer to the next node
```

Example:

```
5 -> 25 -> 75 -> NULL
```

Each number is stored inside one node.

29. Code Explanation: Hash Function

```
int hash(int key) {  
    return key % SIZE;  
}
```

This function converts a key into an index.

If `SIZE = 10`:

```
hash(25) = 25 mod 10 = 5  
hash(122) = 122 mod 10 = 2  
hash(68) = 68 mod 10 = 8
```

The output is always from `0` to `9`.

30. Code Explanation: Initializing the Table

```
struct Node *HT[SIZE];  
  
for (int i = 0; i < SIZE; i++) {  
    HT[i] = NULL;  
}
```

This creates an array of linked-list head pointers.

Each bucket starts as `NULL`.

Simple meaning:

At the beginning, every bucket is empty.

31. Code Explanation: `sortedInsert`

```
void sortedInsert(struct Node **head, int key)
```

This function inserts a key into a linked list while keeping the list sorted.

It receives:

```
head = address of the head pointer  
key = value to insert
```

Why `struct Node **head`?

Because sometimes the head of the linked list must change.

Example:

Before inserting `6`:

```
16 -> NULL
```

After inserting `6`:

```
6 -> 16 -> NULL
```

The head changed from `16` to `6`.

So the function must be able to modify the head pointer.

That is why we pass the address of the head pointer.

32. Why We Use `&table[index]`

In the insert function:

```
void insert(struct Node *table[], int key) {  
    int index = hash(key);  
    sortedInsert(&table[index], key);  
}
```

The important part is:

`&table[index]`

This means:

Give sortedInsert the address of the bucket head pointer.

Why?

Because sorted insertion may need to change the first node of the chain.

Example:

Before:

Index 6: 16 -> NULL

Insert 6.

After:

Index 6: 6 -> 16 -> NULL

The bucket head changed.

So we need `&table[index]`, not just `table[index]`.

33. Code Explanation: Search Function

```
struct Node *search(struct Node *head, int key) {  
    struct Node *current = head;  
  
    while (current != NULL && current->data <= key) {
```



```
        if (current->data == key) {
            return current;
        }
        current = current->next;
    }

    return NULL;
}
```

This function searches inside one chain.

The condition is:

```
current != NULL && current->data <= key
```

This condition supports early stopping.

Example:

Search for 15 in:

```
5 -> 25 -> 75 -> NULL
```

Steps:

```
current = 5
5 <= 15, continue

current = 25
25 <= 15 is false, stop
```

So the function stops early and returns NULL.

34. Code Explanation: Delete Function

```
void deleteKey(struct Node **head, int key)
```

This function deletes a key from a chain.

It uses two pointers:

```
current = the node we are checking  
previous = the node before current
```

Why do we need `previous`?

Because when we delete `current`, the previous node must point to the node after current.

Example:

Before:

```
5 -> 25 -> 75 -> NULL
```

Delete `25`.

`previous` points to `5`.

`current` points to `25`.

After deletion:

```
5 -> 75 -> NULL
```

The link changes from:

```
5 -> 25
```

to:

```
5 -> 75
```

Then the deleted node is freed:

```
free(current);
```

35. Code Explanation: Printing the Table

```
void printTable(struct Node *table[]) {  
    for (int i = 0; i < SIZE; i++) {  
        printf("%d: ", i);  
  
        struct Node *current = table[i];  
  
        while (current != NULL) {  
            printf("%d -> ", current->data);  
            current = current->next;  
        }  
  
        printf("NULL\n");  
    }  
}
```

This prints every bucket.

For each index:

1. Print the index.
2. Walk through the linked list.
3. Print each key.
4. End with NULL.

Example output:

```
5: 5 -> 25 -> 75 -> NULL
```

36. Program Dry Run

The program inserts:

```
16, 12, 25, 39, 6, 122, 5, 68, 75
```

Final table:

```
0: NULL  
1: NULL
```

```
2: 12 -> 122 -> NULL
3: NULL
4: NULL
5: 5 -> 25 -> 75 -> NULL
6: 6 -> 16 -> NULL
7: NULL
8: 68 -> NULL
9: 39 -> NULL
```

Then it searches for 75.

$75 \bmod 10 = 5$

It searches bucket 5:

5 -> 25 -> 75 -> NULL

It finds 75.

Then it deletes 25.

Before deletion:

5 -> 25 -> 75 -> NULL

After deletion:

5 -> 75 -> NULL

37. Complexity Summary

Operation	Average case	Depends on
Hash calculation	$O(1)$	Simple modulus calculation
Insertion	About chain length	How many keys are in the bucket
Successful search	$1 + \text{lambda} / 2$	Usually half the chain
Unsuccessful search	$1 + \text{lambda}$	Usually the whole chain

Operation	Average case	Depends on
Deletion	About chain length	Need to find the key first

Important idea:

Chaining is efficient when the hash function distributes keys evenly.

38. Common Beginner Mistakes

Mistake 1: Forgetting to Initialize Buckets

Wrong:

```
struct Node *HT[10];
```

This creates the array, but the pointers may contain garbage values.

Correct:

```
for (int i = 0; i < 10; i++) {
    HT[i] = NULL;
}
```

Always initialize buckets to `NULL`.

Mistake 2: Confusing Key and Index

A key is the value stored.

An index is the bucket position.

Example:

```
Key = 25
Index = 25 mod 10 = 5
```

So `25` is the key.

`5` is the index.

Mistake 3: Thinking Chaining Removes All Slowness

Chaining solves collisions, but it does not guarantee fast search every time.

If many keys go to one bucket, that chain becomes long.

Then search becomes slow.

Mistake 4: Not Keeping the Chain Sorted

If the phase expects sorted chains, insertion must place keys in order.

Correct:

```
5 -> 25 -> 75
```

Wrong:

```
25 -> 5 -> 75
```

Sorted chains allow early stopping.

Mistake 5: Passing `table[index]` Instead of `&table[index]`

Wrong when the head may change:

```
sortedInsert(table[index], key);
```

Correct:

```
sortedInsert(&table[index], key);
```

Why?

Because insertion may change the first node of the chain.

Mistake 6: Forgetting to Free Deleted Nodes

In C, if memory is created using `malloc`, it should be released using `free`.

Correct:

```
free(current);
```

If you forget this, memory is wasted.

39. Phase 2 Quick Revision Sheet

Main idea

Chaining solves collisions by storing colliding keys in linked lists.

Formula

$$H(x) = x \bmod \text{size}$$

Loading factor

$$\lambda = n / \text{size}$$

Table structure

Hash table = array of linked-list pointers

Empty bucket

NULL

Collision example

$$\begin{aligned} 15 \bmod 10 &= 5 \\ 25 \bmod 10 &= 5 \end{aligned}$$

Both go to bucket `5`:

```
Index 5: 15 -> 25 -> NULL
```

Sorted chain example

```
5 -> 25 -> 75 -> NULL
```

Early stop example

Search for 15 in:

```
5 -> 25 -> 75
```

Stop at 25 because $25 > 15$.

40. Phase 2 Full Summary

Phase 2 teaches open hashing with chaining.

In Phase 1, you learned that collisions happen when many keys map to the same index.

Phase 2 solves that problem by using linked lists.

In chaining, each hash table index points to a linked list.

If a bucket is empty, it contains `NULL`.

If one key maps to a bucket, the bucket points to one node.

If many keys map to the same bucket, the bucket points to a chain of nodes.

The phase uses:

$$H(x) = x \bmod 10$$

When inserting a key, we calculate its index and insert it into the correct chain.

The chains are kept sorted.

Sorted chains allow early stopping during unsuccessful search.

The loading factor is:

```
lambda = n / size
```

In chaining, lambda represents the average chain length if keys are spread evenly.

Chaining is usually efficient when the hash function distributes keys well.

But if the hash function is poor, many keys may go to one bucket, creating a long chain.

Then search can become slow.

Deletion in chaining is done by finding the correct bucket, searching the linked list, unlinking the node, and freeing memory.

The most important idea is:

```
Collision does not mean failure.  
Collision means add the key to the chain at that bucket.
```

41. What You Should Be Able to Explain After Phase 2

After studying Phase 2, you should be able to explain:

1. What chaining means
2. What open hashing means
3. Why chaining uses extra memory
4. What a bucket is
5. Why the hash table is an array of pointers
6. What `NULL` means
7. How linked-list chains store colliding keys
8. Why chains are kept sorted
9. How insertion works in chaining
10. How searching works in chaining
11. Why searching only checks one bucket
12. How early stopping works
13. What loading factor means
14. How to calculate `lambda = n / size`
15. Why a bad hash function creates long chains
16. How deletion works in chaining
17. Why `&table[index]` is used in insertion
18. Why deleted nodes should be freed in C

- 19. Common mistakes in chaining
 - 20. Why Phase 3 is needed after Phase 2
-

42. Mini Practice 1

Use:

```
Table size = 10
H(x) = x mod 10
Keys: 11, 21, 32, 42, 15, 25
```

Calculations:

```
11 mod 10 = 1
21 mod 10 = 1
32 mod 10 = 2
42 mod 10 = 2
15 mod 10 = 5
25 mod 10 = 5
```

Final table:

```
Index 0: NULL
Index 1: 11 -> 21 -> NULL
Index 2: 32 -> 42 -> NULL
Index 3: NULL
Index 4: NULL
Index 5: 15 -> 25 -> NULL
Index 6: NULL
Index 7: NULL
Index 8: NULL
Index 9: NULL
```

43. Mini Practice 2

Use:

```
Table size = 10
H(x) = x mod 10
Keys: 44, 14, 24, 7, 17
```

Calculations:

```
44 mod 10 = 4
14 mod 10 = 4
24 mod 10 = 4
7 mod 10 = 7
17 mod 10 = 7
```

Sorted final table:

```
Index 0: NULL
Index 1: NULL
Index 2: NULL
Index 3: NULL
Index 4: 14 -> 24 -> 44 -> NULL
Index 5: NULL
Index 6: NULL
Index 7: 7 -> 17 -> NULL
Index 8: NULL
Index 9: NULL
```

Note:

Even though 44 was inserted first, the chain at index 4 is sorted:

```
14 -> 24 -> 44
```

44. Final Simple Explanation

Hashing uses a formula to choose where a key should go.

Sometimes different keys choose the same place.

That is called a collision.

Chaining solves the collision by putting all keys with the same index into a linked list.

So instead of losing data, the table stores a chain.

Example:

```
15 mod 10 = 5  
25 mod 10 = 5  
35 mod 10 = 5
```

Final chain:

```
Index 5: 15 -> 25 -> 35 -> NULL
```

That is the heart of Phase 2.

```
Phase 1: Learn how keys become indices.  
Phase 2: Learn what to do when many keys get the same index.
```

The answer is:

```
Open Hashing with Chaining.
```